

Namespaces

Prof. Rizzotti

1 I namespace

Quando scriviamo un programma in C++, utilizziamo variabili, funzioni e altri elementi, ciascuno identificato attraverso un nome.

Ad esempio:

```
1 int numero = 10;
```

In questo caso, **numero** è il nome della variabile.

Quando utilizziamo librerie scritte da altri programmatori, potremmo incontrare funzioni o variabili che hanno lo stesso nome.

Come possiamo distinguerle?

Per risolvere questo problema, C++ mette a disposizione i **namespace**.

Un namespace è un modo efficiente e conveniente per **organizzare e identificare** elementi utili al nostro programma, come funzioni e variabili.

Possiamo immaginare un namespace come un contenitore che raccoglie elementi identificati attraverso un nome.

2 Il namespace std

La libreria standard del C++ utilizza un namespace chiamato:

```
1 std
```

Al suo interno troviamo molti degli elementi che abbiamo già utilizzato nei nostri programmi.

Ad esempio:

- **cout**, per visualizzare informazioni sullo schermo;
- **cin**, per leggere informazioni inserite dall'utente;
- **endl**, per inserire un carattere di nuova riga e svuotare il buffer di output.

Questi elementi appartengono al namespace **std**.

3 L'operatore ::

Per indicare esplicitamente che vogliamo utilizzare un elemento appartenente a un determinato namespace, utilizziamo l'operatore:

```
1 ::
```

Questo operatore prende il nome di **operatore di risoluzione dell'ambito** (scope resolution operator).

La sintassi generale è:

```
1 NAMESPACE_NAME::nome_elemento
```

Ad esempio:

```
1 std::cout
```

indica che vogliamo utilizzare `cout` appartenente al namespace `std`.

Analogamente:

```
1 std::cin
```

indica che vogliamo utilizzare `cin` appartenente al namespace `std`.

3.1 Esempio completo

Consideriamo il seguente programma:

```
1 #include <iostream>
2
3 int main() {
4
5     int numero;
6
7     std::cout << "Inserisci un numero: ";
8     std::cin >> numero;
9
10    std::cout << "Hai inserito: ";
11    std::cout << numero << std::endl;
12
13    return 0;
14 }
```

In questo programma utilizziamo esplicitamente il namespace `std` ogni volta che vogliamo accedere a uno dei suoi elementi.

In particolare:

- `std::cout` permette di visualizzare informazioni;
- `std::cin` permette di leggere un valore inserito dall'utente;
- `std::endl` permette di andare a capo e svuotare il buffer di output.

4 L'istruzione `using namespace`

Scrivere ogni volta il nome completo di un elemento può risultare scomodo.

Per questo motivo, C++ mette a disposizione l'istruzione:

```
1 using namespace NAMESPACE_NAME;
```

Questa istruzione permette di utilizzare gli elementi di un namespace senza dover specificare ogni volta il suo nome.

Possiamo interpretarla intuitivamente come un'indicazione al compilatore:

Se non sai come identificare una certa funzione o variabile, prova a cercarla anche all'interno di `NAMESPACE_NAME`.

In particolare, possiamo scrivere:

```
1 using namespace std;
```

In questo modo, gli elementi del namespace `std` possono essere utilizzati senza dover scrivere ogni volta `std::`.

Il vantaggio principale è che si scrive meno codice.

Lo svantaggio è che il codice diventa molto meno chiaro, soprattutto in programmi più grandi, perché non è immediato capire da quale namespace provengano i vari elementi.

Per questo motivo, durante le lezioni **non utilizzeremo** l'istruzione `using namespace std`; e scriveremo sempre esplicitamente `std::`.

¹

¹Parti di questo documento sono output di LLM.